Surtaal Web System Design

System design documentation for evolving Surtaal from a local desktop-first audio tool into a production-ready web platform without losing the current job-based interaction model.

SCOPE	AUDIENCE	DESIGN GOAL
Web architecture for audio processing, storage, delivery, and operations	Product, engineering, platform, and packaging stakeholders	Support multi-user web delivery for heavy audio workflows such as Demucs stem separation

> **Executive summary.** Surtaal should keep its current split between a light UI and a processing backend, but the backend must be redesigned for distributed execution. The right web architecture is a static frontend, a thin API tier, background workers for audio jobs, object storage for user media, Redis for queueing and ephemeral job state, and Postgres for durable metadata. This design matches Surtaal's existing job model, scales better than request-bound processing, and avoids forcing Demucs and Python audio tooling into the browser.

1. Current State

Surtaal is currently a local two-process application. The React frontend submits multipart requests directly to a FastAPI backend, receives a `job_id`, polls `/job/{job_id}`, and downloads generated files from the backend output directory. Long-running work is started through FastAPI `BackgroundTasks` and executed in-process via `asyncio.to_thread(...)`.

That pattern is a strong starting point for the web because the frontend already thinks in terms of async jobs. However, the current backend keeps uploads, outputs, and job state on a single machine and in memory. That is suitable for a local app, but it breaks down quickly for multi-user web traffic.

Area	Current local design	Why it must change for web
Job state	In-memory Python dictionary	State disappears on restart and cannot be shared across multiple API instances
Uploads and outputs	Local filesystem folders	Not durable across instances and poor fit for multi-node delivery
Heavy processing	Runs inside backend process	Long CPU-bound jobs would starve API capacity and reduce reliability
Client connectivity	Assumes localhost API	Web deployment needs stable public API routing, auth, and tenancy isolation

---

2. Design Drivers

1. Keep the user interaction model simple: upload, process, monitor progress, and download results.
2. Support CPU-heavy and memory-heavy audio jobs without blocking user-facing API requests.
3. Scale from a small MVP to a multi-user deployment without rewriting the entire product.
4. Preserve room for privacy-sensitive positioning by minimizing unnecessary data retention.
5. Stay aligned with the current Python audio stack instead of forcing a premature browser-only rewrite.

---

3. Proposed High-Level Architecture

```
+-------------------+        +--------------------+        +---------------
-------+
| Browser Frontend  | -----> | FastAPI API Tier   | -----> | Redis Queue /
Cache  |
| React + Vite SPA  |        | Auth + job control |        | Job
dispatch/state   |
+-------------------+        +--------------------+        +---------------
-------+
          |                              |                              |
          |                              v                              |
          |                  +-----------------------+                  |
          |                  | Postgres Metadata DB  |                  |
          |                  | Users, jobs, audit    |                  |
```

```
          |                   +-----------------------+                  |
          |                               |                              |
          v                               v                              v
+-------------------+        +--------------------+        +---------------
-------+
| CDN / Static Host |        | Object Storage     | <----- | Audio Worker
Pool    |
| Web assets        |        | Uploads + outputs  |        | Demucs,
ffmpeg, DSP  |
+-------------------+        +--------------------+        +---------------
-------+
```

4. Component Decisions and Rationale

DECISION 1

Use a static web frontend separate from the processing backend

Choice. Host the React frontend as static assets on a CDN-backed platform.

Rationale. The Surtaal UI is already a client-rendered app that communicates with backend endpoints over HTTP. Serving it statically is cheaper, simpler, and easier to scale than coupling page rendering to the audio backend. Static hosting also makes global delivery, cacheability, and zero-downtime deploys much easier.

Why not a server-rendered monolith. Server rendering would not solve Surtaal's core challenge, which is heavy audio processing. It would add deployment complexity without reducing worker load or making Demucs execution easier.

DECISION 2

Keep FastAPI as the API layer

Choice. Retain FastAPI for upload handling, job creation, status lookup, and download orchestration.

Rationale. Surtaal already uses FastAPI successfully, and the current request contracts naturally map to a web API. Reusing FastAPI preserves team familiarity, keeps the audio stack in Python, and avoids unnecessary interface churn between the frontend and backend.

Why not replace it with Node or Go. Those stacks could serve APIs well, but the core processing logic and dependencies are already in Python. Replatforming the API tier would increase the total system boundary count without removing the Python worker dependency.

DECISION 3

Move audio processing into a dedicated worker tier

Choice. Run stem separation, pitch shift, tempo change, mix export, and stitching in workers, not in API processes.

Rationale. Demucs and other DSP steps are long-running and CPU-intensive. An API instance should accept work quickly and stay responsive. Separating workers from the API avoids head-of-line blocking, makes crashes more isolated, and allows scaling compute independently from request throughput.

Why not keep `BackgroundTasks` in the API. That model is acceptable for a single-machine local app, but in web production it causes resource contention, poor autoscaling behavior, and weak failure recovery. Background tasks tied to API lifecycles are not durable enough for multi-minute jobs.

DECISION 4

Use Redis as the queue and ephemeral job-state cache

Choice. Use Redis-backed queueing and transient progress tracking between API and worker nodes.

Rationale. Redis fits Surtaal's need for fast enqueue/dequeue operations, lightweight progress updates, and short-lived state transitions. It reduces the coordination burden between API instances and workers while remaining simple enough for an MVP.

Why not use only Postgres as a queue. Postgres can support jobs, but it is a weaker fit for high-churn, in-flight progress updates and worker heartbeat style interactions. For Surtaal, Redis is the cleaner operational boundary for execution state, while Postgres remains the durable source of truth.

DECISION 5

Use Postgres for durable metadata

Choice. Store users, projects, job history, file references, retention metadata, and audit information in Postgres.

Rationale. Once Surtaal becomes a web product, users expect jobs and generated assets to survive restarts and deployments. Postgres gives durable relational storage, transactional integrity, and a strong base for future features such as billing, project history, or collaboration.

Why not store everything in Redis. Redis is excellent for transient coordination but is a poor long-term home for user-facing history and compliance-relevant metadata.

DECISION 6

Use object storage for uploads and generated outputs

Choice. Store audio inputs and outputs in S3-compatible object storage rather than local disk.

Rationale. Object storage is durable, cheap, and naturally suited to large binary files. It also decouples file delivery from whichever API or worker node happened to process the job. That is the main reason this design scales horizontally without sticky sessions.

Why not keep local files and shared volumes. Shared filesystems add infrastructure complexity and become painful under horizontal scaling. Object storage is the standard, lower-friction answer for this workload shape.

DECISION 7

Preserve the job-based API contract

Choice. Keep submit-job, check-job, and download-result semantics in the public API.

Rationale. The existing frontend already uses this pattern. It maps well to long-running audio work and gives a clean mental model to users. Preserving it reduces UI churn and shortens migration time from local to web deployment.

Why not force synchronous APIs. Synchronous request-response is a poor fit for operations that can run for tens of seconds or minutes. It would increase timeouts and degrade user experience.

DECISION 8

Start with polling, then add push updates later if needed

Choice. Keep frontend polling for job progress in the first web release, with an upgrade path to SSE or WebSockets.

Rationale. Surtaal already polls today, so this keeps the migration low risk. Polling is operationally simple and good enough for a moderate job volume. If the product later needs more immediate multi-job dashboards, server-sent events can be added without changing the underlying execution architecture.

Why not start with WebSockets. WebSockets add infrastructure and session management complexity before there is evidence that polling is insufficient.

DECISION 9

Do not make the browser run the full audio pipeline

Choice. Keep Demucs and heavy DSP on the server side.

Rationale. Surtaal depends on Python-native packages, large compiled libraries, and model inference workloads that are not practical as a mainstream browser-only experience. Browser execution would create inconsistent results across devices, very large client downloads, and poor mobile support.

Where the browser still helps. Interactive playback, waveform views, arrangement UI, and lightweight editing previews can remain client-side.

5. Logical Request Flows

5.1 File upload and job creation

1. User uploads audio from the browser.
2. Frontend requests either a direct upload URL or posts the file to the API.
3. Object storage stores the file, and Postgres records metadata.
4. API creates a job row and enqueues work in Redis.
5. API returns `job_id` immediately.

5.2 Worker execution

1. A worker claims the job from Redis.
2. The worker downloads the input object or streams it into a local temp workspace.
3. The worker runs the relevant Python audio pipeline.
4. The worker writes outputs back to object storage.
5. The worker updates Redis and Postgres with progress, final status, and output references.

5.3 Result retrieval

1. The frontend polls `/jobs/{id}`.
2. When the job is complete, the API returns result metadata and download URLs.
3. The browser downloads from object storage through signed URLs or via the API proxy.

---

6. Data Model and State Boundaries

Data	Home	Reason
Users, auth, projects, retention rules	Postgres	Durable relational data with clear ownership and queryability
Queued jobs, heartbeat, transient progress	Redis	Fast coordination and low-latency state updates
Original uploads and generated media	Object storage	Durability, horizontal scalability, lower cost per GB
Per-job temporary scratch space	Worker local disk	Fast local processing with automatic cleanup after completion

---

7. Deployment Topology

The recommended production topology is three independently deployable services plus managed storage:

1. **Frontend.** Static site on CDN.
2. **API.** FastAPI service behind HTTPS load balancing.
3. **Workers.** Python worker pool on compute-optimized instances.
4. **Managed services.** Redis, Postgres, and object storage.

This topology keeps the control plane small and the compute plane elastic. It also lets Surtaal scale API replicas and workers independently, which is important because a burst of uploads does not imply the same scaling behavior as a burst of long stem separation jobs.

8. Scalability and Capacity Planning

CAPACITY

Scale workers by job class rather than one shared pool forever

Stem separation, tempo change, pitch shift, and mix export have different CPU, memory, and latency characteristics. Over time, Surtaal should classify jobs into queues such as `light-dsp`, `mixing`, and `demucs-heavy`. That prevents small jobs from waiting behind large inference tasks and gives the team more precise autoscaling control.

Initial MVP scaling can still start with one queue and one worker deployment. The classification is a second-phase optimization, not a day-one requirement.

9. Security and Privacy

1. Use authenticated API access for all non-public workflows.
2. Store outputs under tenant-scoped object paths.
3. Prefer signed URLs for downloads rather than permanent public links.
4. Encrypt storage at rest and TLS in transit.
5. Apply strict file size, type, and duration limits at upload time.
6. Run worker jobs in isolated temp directories and clean them after completion.
7. Define retention windows so user uploads do not remain indefinitely without purpose.

The privacy rationale matters especially for Surtaal because its current brand promise is local-first and low-friction. A web version should not casually undermine that trust. The system should make retention and deletion behavior explicit rather than treating media storage as an afterthought.

10. Reliability and Observability

Concern	Recommendation	Why it matters
Job retries	Retry only idempotent or safely restartable jobs	Avoid duplicated outputs and runaway compute costs
Metrics	Track queue depth, job time, failure rate, worker utilization, storage throughput	These are the leading indicators for system stress

Concern	Recommendation	Why it matters
Structured logging	Log with `job_id`, `user_id`, operation type, and worker hostname	Essential for debugging distributed failures
Health checks	Separate readiness for API and worker liveness	Prevents unhealthy workers or cold APIs from serving traffic incorrectly

11. Cost and Product Tradeoffs

A web version of Surtaal introduces ongoing infrastructure cost that the local app does not have. The main cost drivers will be compute time for Demucs, storage of uploaded and generated audio, and network egress for downloads. That means the architecture should be designed with usage controls from the beginning.

- Limit audio duration or file size per plan tier.
- Expire temporary outputs automatically.
- Separate heavy premium workflows from lightweight free workflows.
- Track cost per operation so pricing can eventually reflect actual compute load.

12. Alternatives Considered

Alternative	Why it is tempting	Why it is not the recommended default
Browser-only implementation	No backend hosting cost and easier privacy story	Does not fit Surtaal's current Python and Demucs-heavy stack
Single VM running frontend, API, workers, and local files	Very fast MVP setup	Good pilot option, but weak scaling, durability, and operational isolation
Serverless functions for all processing	Operationally attractive on paper	Long-running audio workloads, native binaries, and model inference fit poorly

Alternative	Why it is tempting	Why it is not the recommended default
WebSockets-first architecture	Real-time feel	Higher complexity without solving the core compute and storage challenges

---

13. Recommended Rollout Plan

Phase 1: Practical MVP

1. Deploy static frontend, FastAPI API, one Redis instance, one Postgres instance, one object store.
2. Run a small worker pool on the same cloud provider.
3. Keep polling-based progress updates.
4. Implement upload limits, basic auth, retention cleanup, and signed downloads.

Phase 2: Operational hardening

1. Split worker queues by job type.
2. Add autoscaling and richer metrics.
3. Introduce project history and user libraries backed by Postgres.
4. Move toward direct-to-object-storage uploads for larger files.

Phase 3: Product expansion

1. Add collaborative workflows and saved projects.
2. Introduce push-based progress updates if product usage justifies it.
3. Offer regional deployment choices if privacy or latency becomes important.

---

14. Final Recommendation

From a system design perspective, Surtaal should not be treated as a conventional CRUD web app. It is a media-processing platform whose central challenge is orchestrating expensive background work safely and predictably. The recommended architecture therefore emphasizes decoupling: static frontend delivery, a thin API control plane, a worker compute plane, durable storage, and clear separation between transient and persistent state.

This is the right design because it preserves the mental model users already have in the local app, aligns with the existing Python audio codebase, and creates a realistic path from MVP deployment to production-scale operation without a full rewrite.

Prepared for the Surtaal repository as system design documentation for a potential web deployment architecture.